BIN702 - Série d'exercices sur le séquençage

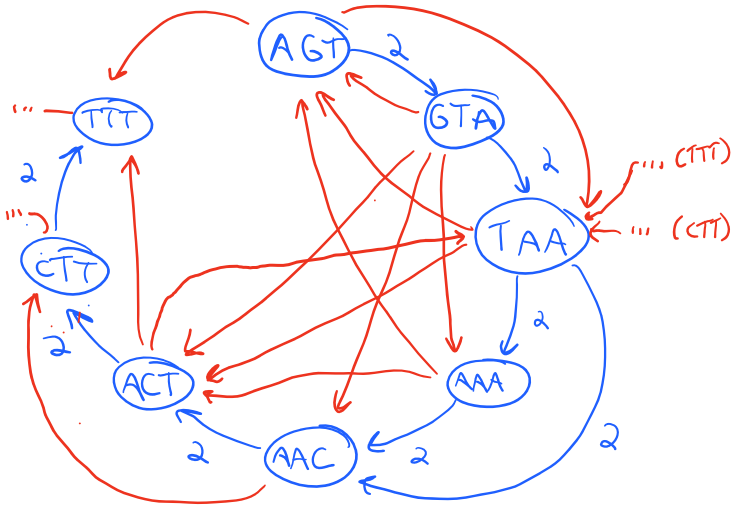
Manuel Lafond

Exercice 1: Soit l'ensemble des 3-mers
$\{AGT, AAA, ACT, AAC, CTT, GTA, TTT, TAA\}$.

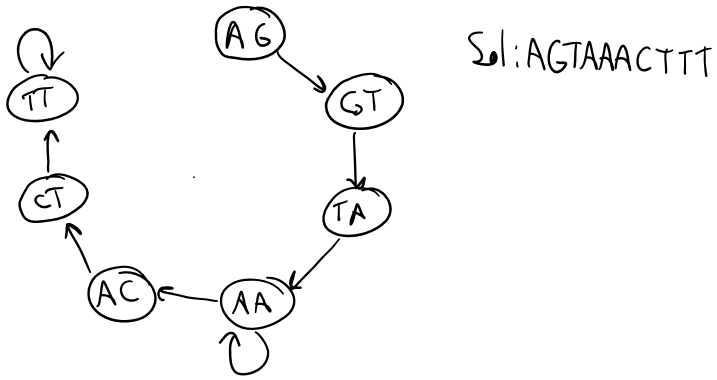
— Construisez le graphe de "overlap" et trouvez la plus courte superséquence à l'aide d'un chemin Hamiltonien.

— Construisez le graphe de De Brujin pour ces 3-mers. Est-ce qu'il existe un chemin Eulérien ? Si oui, comparez la superséquence résultante avec celle obtenue à la question précédente.

Solution.
D'abord, le graphe d'overlap. Cet exercice a été pris dans la référence 2 du plan de cours. Je vois maintenant que c'est un peu sadique de demander de construire TOUT ce graphe. En rouge, les arêtes de poids 1 (si j'en ai oublié, laissez-moi savoir). Une solution optimale prendra toutes les arêtes de poids 2, donc $AGTAAACTTT$.



Voici le graphe de De Brujin. Beaucoup plus facile à reconstruire.



Exercice 2: Donnez un algorithme qui reconstruit un chemin Eulérien dans un graphe, s'il en existe un.

Solution.
Ceci est un algorithme classique sur les graphes. Je vais me contenter de vous donner un lien vers une solution : `https://stackoverflow.com/questions/17467228/looking-for-algorithm-finding-euler-path`.

Exercice 3: On a vu en vidéo que si tous les k-mers d'une séquence sont représentés dans l'entrée, alors la technique du chemin Eulérien ne fonctionne pas. Dans ce cas, on peut essayer de trouver le minimum de chemins tels que chaque arête apparaît exactement une fois sur un des chemins.
Donnez un algorithme qui trouve ce minimum de chemins.
Version facile : faites deux suppositions : (1) supposez que s'il y a un cycle C, alors il existe une solution optimale (i.e. un ensemble de chemins) qui contient C (i.e. il y a un chemin qui suit toutes les arêtes de C de façon consécutive) ; (2) supposez que l'on peut facilement trouver une solution optimale si vous avez un graphe orienté acyclique. Avec ces deux suppositions,
Version difficile : démontrez ou infirmez les deux suppositions ci-haut. Sont-elles vraies ou fausses ?

Solution.
Version facile. Avec les suppositions, on peut faire la chose suivante. L'idée est que s'il y a un cycle C, on l'enlève, on trouve une solution récursivement, puis on réintègre C dans un chemin de la solution.

```
fonction getMinChemins(G)
    si G ne contient pas de cycle alors
        Trouver un nombre minimum de chemins 𝒞 qui contient chaque
          arête. Ceci est faisable en temps polynomial selon la
          supposition (2) ;
        Retourner 𝒞;
    sinon
        Trouver un cycle C;
        Soit v un sommet de C qui a des arêtes en dehors de C;
        //si v n'existe pas, on G est un ensemble de cycles disjoints.
          On pourrait traiter ce cas trivial séparément;
        Soit G' obtenu à partir de G en retirant chaque arête de C;
        𝒞 = getMinChemins(G');
        Soit P ∈ 𝒞 un chemin contenant v;
        Obtenir P' en remplaçant v par le cycle C;
        Retourner (𝒞 \ {P}) ∪ {P'};
    fin
```

Pour être précis, si on a un cycle $C = (v_1, v_2, \ldots, v_k, v_1)$ et un chemin $P = (p_1, p_2, \ldots, p_i = v_1, p_{i+1}, \ldots, p_l)$ contenant v_1, remplacer v_1 par C dans P créé un nouveau chemin $P' = (p_1, p_2, \ldots, p_i = v_1, v_2, \ldots, v_k = p_i, p_{i+1}, \ldots, p_l)$.
Si on avait une solution optimale pour G', le remplacement de v par C dans P ne créé pas de nouveau chemin. Donc, la seule raison pour laquelle cet algorithme serait sous-optimal pour G serait si les arêtes de C n'étaient pas consécutives dans cette solution optimale. Mais avec la supposition (1), cet algorithme donne une solution optimale.

Pour ce qui est de la version difficile, je n'entre pas dans les détails et je vous laisse en suspens. Les deux suppositions devraient être vraies, mais les preuves deviennent un peu laborieuses. De plus, la réponse ne se trouve pas dans la littérature, étonnamment. L'idée de (2) est de faire un tri topologique (`https://fr.wikipedia.org/wiki/Tri_topologique`), de démarrer un chemin à partir du sommet minimal de degré non-zéro et de suivre les arêtes de façon gloutonne jusqu'à ce qu'on bloque. Ceci forme notre premier chemin, on retire ses arêtes et on répète jusqu'à ce que toutes les arêtes soient incluses.
Pour démontrer (1) formellement, il faut prendre une solution optimale et la modifier pour qu'elle inclue notre cycle C sans augmenter le nombre de chemins. Si vous êtes trop curieux ou curieuses et que vous voulez une démonstration, venez me voir en personne !

Exercice 4: Soit $S = ACATACACAG$. Construisez la transformée Burrows-Wheeler de S. Montrez ensuite les intervalles qui seront considérés lors de la recherche du mot ACA.

Solution.
Ajoutons un caractère de terminaison \$, pour ainsi obtenir $S = ACATACACAG\$$. En triant les rotations, on obtient

```
$ACATACACAG
ACACAG$ACAT
ACAG$ACATAC
ACATACACAG$
AG$ACATACAC
ATACACAG$AC
CACAG$ACATA
CAG$ACATACA
CATACACAG$A
G$ACATACACA
TACACAG$ACA
```

En prenant la dernière colonne, la transformée est donc $GTC\$CCAAAAA$.

Exercice 5: Supposez que vous avez déjà le tableau de suffixes d'une chaîne S. Montrez comment construire la transformée de Burrows-Wheeler (i.e. la dernière colonne des rotations) à partir du tableau.

Solution.
Réponse courte :
le tri des suffixes est le même que le tri des rotations. On peut donc avoir le dernier caractère de la i-ème rotation en prenant le caractère précédant la position du i-ème suffixe.

Réponse plus détaillée :
Soit SA le tableau de suffixes pour S. Rappelons que l'on suppose que $S[n] = \$$ est le caractère de terminaison, et que $\$ < c$ pour tout caractère c. Rappelons aussi que dans SA, $SA[i]$ contient la position du i-ème suffixe de S selon l'ordre lexicographique.

La seule différence entre le suffixes à la position i et la rotation à la position i est que le suffixe est égal à $S[i..n]$ et la rotation est égale à $S[i..n]S[1..i-1]$. Puisque $S[n] = \$$, il s'avère que de trier les suffixes est équivalent à trier les rotations (plus précisément, en utilisant le fait que $\$ < c$ pour tout $c \in \Sigma$, on a que la rotation de S débutant à $SA[i]$ est lexicographiquement plus petite que la rotation débutant à $SA[i+1]$, et ce pour tout $i = 1..n-1$).
Donc, le i-ème caractère de la transformée Burrows-Wheeler est le dernier caractère de la rotation débutant à la position $SA[i]$. Ce caractère est $S[SA[i]-1]$ (où $S[0]$ est défini comme $S[n] = \$$). Il devient donc facile de construire la transormée en temps $O(n)$.